

MultiAgentBench: LLM 에이전트의 협업과 경쟁 평가

짧은 요약은 </notes/multiagentbench-llm-agents/>에서 볼 수 있다.

원문: [MultiAgentBench: Evaluating the Collaboration and Competition of LLM agents](#) — Zhu et al., University of Illinois Urbana-Champaign, arXiv 2503.01935v1

아래 글은 원문 논문의 순서와 서술을 최대한 따라가며 한국어로 옮긴 것이다. 긴 부록은 원 구조를 유지하되, 환경·지표·프롬프트·사례의 핵심 내용을 한국어로 종합했다.

PDF 다운로드

- [한국어 번역본 PDF](#) — 한국어 번역·종합 PDF판
- [arXiv 원문 PDF](#)

초록

대규모 언어 모델(large language models, LLMs)은 자율 에이전트로서 인상적인 역량을 보였지만, 기존 벤치마크는 단일 에이전트 과제에 치우치거나 좁은 도메인에 제한되어 다중 에이전트 조정과 경쟁의 동역학을 충분히 포착하지 못했다. 이 논문은 다양한 상호작용 시나리오에서 LLM 기반 다중 에이전트 시스템을 평가하는 종합 벤치마크 MultiAgentBench를 소개한다. 제안 프레임워크는 과제 완성도뿐 아니라 milestone 기반 KPI로 협업과 경쟁의 품질을 측정하고, star, chain, tree, graph topology 같은 조정 프로토콜과 group discussion, cognitive planning 같은 전략을 비교한다. 주요 결과로는 GPT-4o-mini가 평균적으로 높은 task score를 보이고, research 시나리오에서 graph 구조가 조정 프로토콜 중 강하며, cognitive planning이 milestone achievement rate를 3% 개선한다는 점이 보고된다.

1. 서론 (Introduction)

GPT-3 [Brown2020GPT3], GPT-4 [OpenAI2023GPT4], Gemini [team2023gemini], DeepSeek-R1 [guo2025deepseek] 같은 대규모 언어 모델(large language models, LLMs)은 이제 인간과 유사한 언어 이해와 생성을 보이며, 환경, 도구, 다른 에이전트와 상호작용하는 자율 에이전트(autonomous agents)로 활용될 수 있게 되었다 [Wang2023CommunicativeAgents, Park2023GenerativeAgents, OpenAI2023FunctionCalling]. 단일 에이전트(single-agent) 시스템도 인상적인 역량을 보여 주었지만, 독립적으로 작동할 경우 과제 수행 효율이 제한될 수 있고 복잡하며 사회적으로 동적인 시나리오를 시뮬레이션하기에는 부족하다.

반대로 다중 에이전트(multi-agent) 구성 [Li2023ChatDev, Wang2023Roleplaying, unleashing2024]은 여러 LLM 기반 에이전트가 협업(collaborate), 조정(coordinate), 공동 계획(jointly plan)을 수행하도록 하여 이러한 한계를 다룬다. 이 패러다임은 과제 수행 효율을 높일 뿐 아니라 사회적 상호작용과 동역학을 더 현실적으로 시뮬레이션할 수 있게 한다. 그 결과 전략적 의사결정(strategic decision-making), 게임 [Silver2017Mastering], 소프트웨어 개발 [Wang2023CommunicativeAgents] 같은 영역에서 성능을 향상시킬 수 있다.

그림 1 개요: MultiAgentBench 평가 과정은 다양한 상호작용 환경에서 다중 에이전트 시스템 조정을 평가하며, 특히 과제 성과와 조정에 초점을 둔다.

LLM 역량이 크게 발전했음에도, 현재의 평가 패러다임은 다중 에이전트 시나리오를 충분히 다루지 못한다. AgentBench [liu2023agentbench], VisualAgentBench [Sun2023VisualAgentBench], GAIA [mialon2023gaia],

ToolBench [qin2024toolllm], HumanEval [Chen2021Evaluating] 같은 전통적인 단일 에이전트 벤치마크는 주로 고립된 추론(reasoning)과 생성(generation)에 초점을 두며, 다중 에이전트 상호작용에 내재한 동역학을 간과한다.

이 공백을 해결하기 위해 저자들은 **MultiAgentBench**를 제안한다. MultiAgentBench는 넓은 범위의 과제 해결(task-solving) 및 시뮬레이션(simulation) 시나리오에서 LLM 기반 다중 에이전트 시스템을 평가하도록 설계된 포괄적 벤치마크다. 주요 장점은 다음과 같다.

1. 다중 도메인 평가(multi-domain evaluation): 협업 코딩(collaborative coding)부터 게임까지 다양한 영역을 포함해 실제 적용 가능성을 넓힌다.
2. 조정과 경쟁 포착(capturing coordination and competition): 기존 단일 에이전트 벤치마크와 달리, 조정 동역학(coordination dynamics)과 경쟁적 상호작용(competitive interactions)을 명시적으로 측정하여 다중 에이전트 환경의 고유한 어려움을 드러낸다.
3. 맞춤형 지표와 유연한 프로토콜(tailored metrics and flexible protocols): 마일스톤 진행 상황과 개별 기여를 추적하는 핵심 성과 지표(Key Performance Indicator, KPI)를 포함한 새로운 지표를 제안하여 계획 품질(planning quality)과 의사소통 효과(communication effectiveness)를 체계적으로 평가한다. 또한 프레임워크인 MARBLE(Multi-agent coordination Backbone with LLM Engine)은 별(star), 체인(chain), 트리(tree), 그래프 기반(graph-based) 접근 같은 여러 의사소통 토폴로지(communication topologies)를 지원하고, 다양한 추론 전략(reasoning strategies)을 수용한다.

저자들이 요약한 기여는 다음과 같다.

1. **MultiAgentBench**와 **MARBLE** 프레임워크를 도입한다. 이는 여섯 가지 다양한 상호작용 시나리오에서 LLM 기반 다중 에이전트 시스템을 엄밀하게 평가하며, 협력적 동역학과 경쟁적 동역학을 모두 포착한다. 특히 인지적 계획(cognitive planning) 기능은 마일스톤 달성률을 3% 향상시킨다고 보고한다.
2. 과제 성공뿐 아니라 조정 품질(coordination quality)도 평가하는 혁신적 평가 지표를 제안한다. 여기에는 마일스톤 기반 KPI, 구조화된 계획 및 의사소통 점수, 그리고 상충 목표 과제(conflicting-goal tasks), 내부 성능 지표, 계획과 의사소통의 경쟁적 측면을 포착하는 전용 경쟁 점수(competition score)가 포함된다.
3. 실험은 다중 에이전트 조정에서 일종의 “aha-moments”를 드러낸다. 즉 에이전트들이 창발적 사회 행동(emergent social behaviors)을 보이기 시작하며, AGI 수준 협업을 향한 유망한 통찰을 제공한다고 저자들은 설명한다 [feng2024how].

2. 관련 연구 (Related Work)

그림 2 개요: MARBLE은 조정 엔진(coordinate engine)과 인지 모듈(cognitive module)을 통해 과제 정보, 페르소나 데이터, 도메인 데이터베이스, 메모리 모듈, 환경 사이의 상호작용을 보여 준다.

2.1 LLM 기반 다중 에이전트 시스템 (LLM-Based Multi-Agent Systems)

LLM 기반 다중 에이전트 시스템은 여러 영역에서 협업적 문제 해결(collaborative problem-solving)을 가능하게 했다 [Park2023GenerativeAgents, li2023camelcommunicativeagentsmind, chen2023agentversefacilitatingmultiagentcollaboration]. 이러한 시스템은 문헌 검토와 실험 설계를 통해 과학 연구를 지원하고 [zhou2024hypothesis, agarwal2024litllmtoolkitscientificliterature], 코드 생성과 유지보수를 포함한 소프트웨어 공학 과제를 수행하며 [huang2023agentcoder, wu2023autogen, zhou2023agents, hong2024metagpt, ishibashi2024self,

islam2024mapcoder, openhands, zhugegptswarm, bouzenia2024repairagent], 게임 응용에도 활용된다 [chen2023gamegpt].

Minecraft 환경에서는 에이전트가 건설부터 내비게이션까지 복잡한 과제를 수행한다 [wang2023voyageropenendedembodiedagent, chen2023agentversefacilitatingmultiagentcollaboration, yu2024minelandsimulatinglargescalemultiagent, dong2024villageragentgraphbasedmultiagentframework]. GameNGen은 DOOM에서 실시간 상호작용을 가능하게 하며 [valeveski2024diffusion], CUISINEWORLD는 다중 에이전트 협업을 벤치마크한다 [gong2023mindagent]. 응용 범위는 사회적 추론 게임 (social deduction games), 게임 이론 (game theory) [xu2023magic], 의료 [ke2024enhancing, kim2024adaptive], 비즈니스 [chen2024agentverse], 교육 [gosling2024multi], 도시 계획 [zhou2024largelanguagemodelparticipatory]까지 확장된다.

그러나 진전에도 불구하고 효과적 의사소통, 창발 행동, 확장성 (scalability)은 여전히 과제로 남아 있다 [agashe2024llmcoordinationevaluatinganalyzingmultiagent]. 이는 다중 에이전트 조정을 평가하기 위한 강력한 평가 프레임워크가 필요함을 뒷받침한다.

2.2 다중 에이전트 협업 (Multi-Agent Collaboration)

최근 다중 에이전트 시스템 연구는 두 가지 상호보완적 확장 패러다임 (scaling paradigms)을 강조한다. 하나는 에이전트의 추론과 적응성을 강화하는 인지적 확장 (cognitive scaling)이고, 다른 하나는 대규모 에이전트 집단을 활용해 창발 행동을 이끌어내는 개체군 확장 (population scaling)이다 [zhugegptswarm, qian2024scalinglargelanguagemodelbasedmultiagentcollaboration].

인지적 확장은 동적 아키텍처 적응 (dynamic architecture adaptation), 자기조직적 조정 전략 (self-organizing coordination strategies) 같은 메커니즘을 탐구하여 에이전트 의사소통의 가장 효과적인 패턴을 찾는다 [zhugegptswarm]. 반면 개체군 기반 확장은 더 많은 에이전트가 계층적 위임 (hierarchical delegation), 탈중앙 합의 (decentralized consensus) 등 다양한 협업 패턴을 통해 집단적으로 상호작용할 때 비선형적 성능 향상 (nonlinear performance gains)이 나타날 수 있음을 보여 준다 [qian2024scalinglargelanguagemodelbasedmultiagentcollaboration].

이러한 접근은 지정학적 갈등 시뮬레이션 (geopolitical conflict simulation) [hua2024warpeacewaragentlarge] 부터 과학적 발견 워크플로 (scientific discovery workflows) [zhou2024hypothesis, zhang2025crewfacilitatinghumanaiteaming] 까지 복잡한 응용을 가능하게 한다.

3. 방법론 (Methodology)

3.1 프레임워크 설계

저자들이 제안하는 평가 프레임워크 MARBLE은 다중 에이전트 조정 시스템을 평가하기 위한 실행 프레임워크다. 이 프레임워크는 적응적 협업 (adaptive collaboration), 효율적 의사소통 (efficient communication), 전략적 과제 실행 (strategic task execution)을 가능하게 하는 여러 모듈을 연결한다. 중심에는 **Coordination Engine**이 있으며, 이 엔진은 Agent Graph, Cognitive Module, 그리고 실행을 조정하는 엔진 구성요소를 초기화하고 동기화한다.

Agent Graph Module은 설정 데이터를 구조화된 그래프 $G = (\mathcal{A}, E)$ 로 변환한다. 여기서 $\mathcal{A} = \{a_1, a_2, \dots, a_n\}$ 는 에이전트 집합이고, 각 edge는 (a_i, r, a_j) 형태의 triple이다. 관계 r 은 협업 (collaborates), 감독 (supervises), 협상 (negotiates)처럼 에이전트 사이의 상호작용 유형을 나타낸다. 이 설계의 의도는 모든

에이전트가 무작위로 대화하는 것이 아니라, 명시적으로 정의된 관계를 가진 에이전트끼리만 의사소통하고 조정하도록 만드는 것이다. 이는 실제 조직의 협업, 위계, 협상 구조를 모사한다.

Cognitive Module은 에이전트의 책임 있는 진화(responsible agent evolution)와 사회적 지능(social intelligence)을 담당한다. 각 에이전트의 페르소나(persona), 에이전트 간 관계, Chain-of-Thought나 ReACT 같은 추론 전략, 환경 관찰, 메모리 상태를 통합하여 내부 상태를 갱신한다. 저자들은 이를 인간의 마음이론(theory of mind)과 사회적 단서 기반 추론에 비유한다. 에이전트는 과거 경험과 사회적 관계를 바탕으로 상대의 의도와 역할을 추론하고, 예상 결과와 실제 결과의 차이를 통해 다음 계획을 조정한다.

Coordination Engine은 전체 실행 흐름을 관리한다. 설정 모듈을 통해 에이전트, 과제, 관계를 초기화하고 Agent Graph를 구성한 뒤, 환경과 도구 기능을 준비한다. 저자들은 프레임워크 안의 역할을 크게 planner와 actor로 나눈다. Planner는 과제 입력을 설계하고, 전략을 세우며, 과제 할당을 관리한다. Actor는 Agent Graph 안에서 실제 행동을 수행하며, 도구를 통해 환경 또는 다른 에이전트와 상호작용한다.

3.2 조정 프로토콜과 계획 전략

MARBLE은 네 가지 조정 프로토콜을 지원한다: star, tree, graph-mesh, chain.

- Star는 단일 중앙 planner가 모든 actor에게 과제를 할당하고 피드백을 통합하는 중앙집중형 구조다. 강한 감독이 필요한 과제에 유리하지만, 규모가 커질수록 병목이 생길 수 있다.
- Tree는 계층적 중앙집중 구조다. 최상위 planner가 하위 planner에게 위임하고, 하위 planner가 다시 actor를 조정한다. 중앙 통제와 확장성을 절충한다.
- Graph-mesh는 여러 actor가 직접 연결되어 병렬적으로 의사소통하고 분산 의사결정을 수행하는 탈중앙 구조다. 동시적 계획과 유연한 정보 흐름에 강하다.
- Chain은 actor가 순차적으로 연결되어 앞선 에이전트의 결정을 다음 에이전트가 이어받는 구조다. 순차 의존성이 있는 과제에는 맞지만 병렬 처리는 제한될 수 있다.

중앙집중형 프로토콜에서 planner는 네 가지 계획 방식을 사용할 수 있다. Vanilla prompt는 직접적인 zero-shot 지시로 계획을 생성한다. CoT 방식은 과제, 에이전트 프로필, 이전 하위 과제 요약을 입력받아 단계별 추론을 유도한다. Group discussion은 여러 에이전트가 제약과 관점을 공유하게 해 계획을 다듬는다. Cognitive self-evolving planning은 Reflexion과 유사하게 각 과제의 기대 결과와 진행 상황을 메모리에 저장한 뒤, 이후 실제 성과와 비교해 경험을 생성하고 다음 계획을 조정한다.

3.3 벤치마크 설계

MultiAgentBench는 과제 지향(task-oriented) 환경과 사회 시뮬레이션(social-simulation-based) 환경을 모두 포함한다. 저자들은 기존 다중 에이전트 과제나 데이터셋을 각색한 시나리오와, LLM이 생성하고 사람이 검증·수정한 시나리오를 결합했다. 이 방식은 기존 과제에서 오는 현실성과 생성 기반 확장에서 오는 다양성을 동시에 확보하려는 설계다.

공통 목표를 가진 에이전트 시나리오에는 네 가지 대표 과제가 있다. Research task는 ResearchTown 설정을 따라, 보완적 연구 프로필을 가진 에이전트들이 특정 주제의 새 연구 제안서를 공동 작성한다. Minecraft building task는 공유 환경에서 블록 구조물을 협력해 만든다. Database error analysis는 정확히 다섯 에이전트가 각기 다른 시스템 불일치 원인을 진단한다. Coding challenge는 공동 문제 해결과 소프트웨어 모듈 개발을 요구한다. 각 과제는 연구 주제, Minecraft 구조물, 데이터베이스 오류, 코딩 목표를 달리해 100개 테스트 사례로 확장된다.

상충 목표를 가진 사회 시뮬레이션에는 Werewolf와 Bargaining이 포함된다. Werewolf에서는 두 진영이 속임수와 추론을 활용해 대립하고, Bargaining에서는 공유 자원과 가격 조건을 둘러싼 협상을 통해 개별 이익을 극대화하려 한다. 두 설정은 불확실성 아래의 적응성, 갈등 해결, 협상 능력을 평가한다.

각 시나리오는 distinct agent roles와 graph relationships를 갖는다. 예를 들어 프로젝트 관리자, 도메인 전문가, 기술 전문가 같은 역할을 나누고, star, tree, chain, mesh 관계를 통해 정보 공유와 의사결정 방식을 제약한다.

3.4 마일스톤과 평가 지표

MARBLE의 반복 실행을 평가하기 위해 각 과제는 유연한 milestone $m_1, m_2, \dots$ 으로 나뉜다. 연구 과제에서는 예를 들어 연구 제안서의 5Q를 완성하거나 기존 5Q를 개선하는 것이 milestone이 될 수 있다. 실행 중 LLM 기반 detector가 각 milestone이 달성되었는지 감시하고, 결과와 기여 에이전트를 기록한다.

Task Completion Metrics는 milestone 기반 KPI와 최종 산출물 품질을 함께 본다. 각 에이전트 j 가 기여한 milestone 수를 n_j , 전체 milestone 수를 M , 에이전트 수를 N 이라고 할 때 전체 KPI는 다음과 같이 정의된다.

$$\text{KPI}_{\text{overall}} = \frac{1}{N} \sum_{j=1}^N \text{KPI}_j = \frac{1}{NM} \sum_{j=1}^N n_j.$$

별도의 task-based score도 계산한다. 연구와 협상 과제는 LLM이 정의한 rubric을 사용하고, Minecraft, Werewolf, database error fix, coding 같은 과제는 accuracy나 hit rate 등 rule-based metric을 사용한다.

Coordination Metrics는 에이전트의 의사소통과 계획 능력을 측정한다. Communication Score C_{score} 는 과제 설명, 에이전트 프로필, 집계된 커뮤니케이션 데이터를 입력으로 받아 5점 척도로 평가하며, 커뮤니케이션이 없으면 0점이다. Planning Score P_{score} 는 과제 조직화, 역할 유지, 전략 적응을 평가한다. 최종 Coordination Score(CS)는 두 점수의 평균이다. 저자들은 프롬프트 기반 평가가 인간 평가와 얼마나 맞는지 부록에서 비교한다.

4. 실험 (Experiment Setup and Results)

4.1 실험 설정

MARBLE은 함수 호출(function calling) 능력을 필요로 하므로, 저자들은 오픈소스 모델 세 개와 폐쇄형 모델 두 개를 평가한다. 오픈소스 모델은 Meta-Llama-3.3-70B, Meta-Llama-3.1-70B-Instruct-Turbo, Meta-Llama-3.1-8B-Instruct-Turbo이고, 폐쇄형 모델은 GPT-3.5-turbo-0125와 GPT-4o-mini다. 오픈소스 모델은 Together AI 서비스를 통해 기본 파라미터로 접근했다.

Agent action에는 최대 토큰 수 1024, temperature 0.7, top_p 1.0을 사용했다. 연구 과제의 최대 iteration은 5, Minecraft의 최대 iteration은 20으로 설정했다. 최대 커뮤니케이션 iteration도 5이고, 각 에이전트의 장기 base memory는 제한 없이 둔다. 메인 실험에서는 graph-mesh 조정 프로토콜을 사용했다. 평가는 Task Score(TS)와 Coordination Score(CS) 두 축으로 진행된다.

4.2 모델별 시나리오 성능

다섯 모델을 여섯 시나리오에서 비교한 결과, 저자들은 기본 모델 능력이 여전히 task completion의 핵심 요인이라고 해석한다. GPT-4o-mini는 여러 과제에서 높은 Task Score를 보인다. Research에서는 TS 84.13%로 Meta-Llama-3.1-8B(80.87%)와 Meta-Llama-3.1-70B(80.80%)를 앞선다. Coding에서도 GPT-4o-mini는 TS 65.10으로 비교 모델보다 높고, Bargaining에서도 TS 74.47과 CS 74.20으로 가장 강한 조합을 보인다.

반면 Coordination Score가 항상 높은 Task Score로 이어지지 않는다. 대표적으로 Minecraft에서 Meta-Llama-3.1-70B는 CS 75.00으로 높지만 TS는 0.21에 불과하다. 저자들은 이 불일치를 모델의 도구 실행·함수 호출 역량 부족과 연결한다. 즉 조정은 성능에 기여하지만, 기본 실행 능력의 결핍을 보상하지는 못한다. Meta-Llama-3.3-70B는 Research CS 72.00과 WereWolf TS 36.33, CS 76.30에서 강점을 보였지만, 여러 과제의 TS는 GPT-4o-mini보다 낮다. 따라서 단일 지표만으로 모델의 다중 에이전트 적합성을 판단하기 어렵고, task-specific ability와 coordination ability를 함께 봐야 한다.

표의 주요 수치는 다음과 같이 정리할 수 있다. Research TS는 GPT-4o-mini가 84.13으로 최고이고, Research CS는 Meta-Llama-3.3-70B가 72.00으로 최고다. Minecraft TS는 GPT-4o-mini가 33.60으로 최고이나 Minecraft CS는 Meta-Llama-3.1-70B가 75.00으로 최고다. Database TS는 Meta-Llama-3.1-70B가 53.00으로 최고이고, Database CS는 GPT-3.5-turbo가 60.89로 최고다. Coding TS는 GPT-4o-mini가 65.10, Coding CS는 GPT-3.5-turbo가 76.20으로 가장 높다. WereWolf에서는 Meta-Llama-3.3-70B가 TS 36.33과 CS 76.30 모두에서 최고다.

4.3 협업 프로토콜과 계획 전략의 효과

Research 시나리오에서 star, tree, graph, chain 프로토콜을 비교한 결과, graph 기반 프로토콜이 task performance, planning efficiency, token usage 면에서 가장 강한 결과를 보였다. Star와 graph는 task score가 유사하지만, tree는 높은 token usage와 낮은 task·coordination score로 좋지 않았다. 이는 계층 구조가 항상 효율적인 것은 아니며, 과제 성격에 따라 정보 흐름의 병목과 관리 비용이 성능을 해칠 수 있음을 시사한다.

계획 prompt 전략 비교에서는 Cognitive Evolving Planning이 coordination 면에서 가장 우수했고, task score도 CoT와 비슷한 수준을 달성했다. 흥미롭게도 Group Discussion은 모든 지표에서 가장 낮았다. 저자들은 너무 많은 에이전트가 계획 그룹에 참여하면 실제 조직에서처럼 조정 비용이 커져 비효율적일 수 있다고 해석한다.

4.4 절제 연구

첫 번째 절제 연구는 Minecraft에서 최대 iteration 수를 바꾸는 실험이다. GPT-4o-mini를 대상으로 10개 Minecraft 과제를 평가하고, 여섯 가지 최대 iteration 설정을 비교했다. Task score와 coordination score는 1에서 7 iteration까지 증가하지만, 10 iteration에서 급격히 떨어진다. 20 iteration에서는 task score가 일부 회복되지만, coordination score는 7 iteration 이후 거의 변하지 않는다. 저자들은 어려운 과제에서 과도한 반복이 커뮤니케이션 오버헤드나 상충 지시를 유발해 조정을 악화시킬 수 있다고 본다.

두 번째 절제 연구는 Research 시나리오에서 에이전트 수를 1, 3, 5, 7로 바꾸는 실험이다. 저자들은 주저자가 최소 7명인 논문 20개를 선택했다. 에이전트 수가 증가할수록 전체 KPI는 감소하는데, 이는 협업 복잡성이 커질수록 개별 기여의 효율이 떨어지는 trade-off와 맞는다. 그러나 1명에서 3명으로 늘릴 때 average coordination score는 크게 개선되고 task score도 완만히 증가한다. 즉 적당한 규모 확장은 조정 효율을 높일 수 있지만, 그

이상은 추가 조정 비용이 성능 이득을 상쇄할 수 있다.

4.5 창발 행동 분석

MultiAgentBench에서 목표 지향 창발 행동(goal-driven emergent behaviors)은 팀 조정의 중요한 신호다. 저자들은 이를 “aha-moments”라고 부르며, 개별 에이전트가 공유 목표에 맞춰 행동을 정렬하는 순간이 새로운 조정 전략과 적응적 집단 지능을 촉발한다고 주장한다.

정보 비대칭과 역할 갈등 아래에서 세 가지 패턴이 관찰된다. 첫째, 전략적 정보 공유(strategic information sharing)다. 에이전트는 신뢰와 맥락에 따라 핵심 정보를 선택적으로 공개한다. 예를 들어 Werewolf에서 예언자나 마녀가 검사 결과나 아이템 사용 계획을 지나치게 숨기면 팀 전체가 실패할 수 있다. 둘째, 신뢰 양극화 협업(trust-polarized collaboration)이다. 지나치게 의심 많은 주민은 같은 편을 공격할 수 있고, 늑대인간은 거짓 합의를 만들어 혼란을 이용할 수 있다. 셋째, 역할 기반 전략 반복(role-driven strategy iteration)이다. 예언자와 마녀 같은 역할은 게임이 진행되면서 보수적 전략에서 리더십 또는 위험 감수 전략으로 이동할 수 있다. 이는 역할과 목표가 에이전트의 의사결정 방식을 계속 갱신한다는 점을 보여 준다.

5. 결론

본 연구는 LLM 기반 멀티 에이전트 시스템을 다양한 상호작용 시나리오에서 평가하기 위한 종합 벤치마크인 MultiAgentBench와 MARBLE 프레임워크를 제안한다. 제안한 평가 지표는 단순한 과제 성공 여부를 넘어, 구조화된 계획 수립, 커뮤니케이션 점수, 경쟁 기반 평가를 통해 조정 품질을 포착한다. 실험 결과는 핵심적인 창발적 사회 행동을 보여 주며, 향후 멀티 에이전트 연구에 유용한 통찰을 제공한다.

6. 한계

제안한 멀티 에이전트 벤치마크와 프레임워크는 다양한 과제와 평가 지표를 제공하지만, 적용 가능성과 강건성을 높이기 위해 다음 영역은 추가 탐구가 필요하다.

시나리오와 모델 범위 확장

현재 벤치마크는 연구 공동 저술, Minecraft 건축, 데이터베이스 오류 분석, 코딩 협업, 일부 경쟁 시나리오(예: Werewolf, bargaining)에 초점을 둔다. 실제 멀티 에이전트 상호작용의 복잡성을 더 잘 포착하려면, 향후 연구는 오픈 월드 환경, 더 풍부한 사회적 인지가 필요한 시나리오, 과업 지향 대화 같은 응용 측 과제를 포함할 수 있다. 모델 측면에서도 본 연구는 전체 스펙트럼을 포괄하지 않는다. 향후에는 DeepSeek 모델 계열 등 최신 모델의 결과를 포함할 수 있다.

절제 연구 강화

현재 분석은 주로 전반적인 조정 성능과 경쟁 성능에 초점을 맞추며, 특정 구성 요소에 대한 세분화된 통찰은 상대적으로 덜 탐구되었다. 향후 실험은 장기 기억, 단기 기억, 공유 기억 같은 서로 다른 기억 메커니즘과 다양한 멀티 에이전트 워크플로 방법에 초점을 맞출 수 있다.

경쟁 메커니즘 고도화

벤치마크에는 경쟁 과제가 포함되어 있지만, 다자 협상, 반복적 전략 플레이, 확률적 요소가 있는 실제 멀티 에이전트 상호작용의 복잡성을 완전히 포착하지는 못한다. 변화하는 환경에서 에이전트가 협력적 역할과 적대적 역할 사이를 어떻게 전환하는지 조사하는 것은 유망한 방향이다.

개방형·불명확 과제 처리

프레임워크의 대부분 과제는 연구 제안서 완성이나 데이터베이스 불일치 해결처럼 목표가 잘 정의되어 있다. 그러나 실제 응용에서는 명확한 성공 기준이 없는 개방형 또는 모호한 맥락에서 에이전트가 작동해야 하는 경우가 많다. 향후 확장은 멀티 에이전트 시스템이 탐색적이고 목표가 명시되지 않은 시나리오에 어떻게 적응하는지 다룰 수 있다.

7. 부록 개요

7.1 Contributions

부록은 각 저자의 기여를 명시한다. Kunlun Zhu는 팀 리드로서 메인 코드베이스 기본 설계, 연구 환경, 조정 엔진, 평가기 기본 설계, 본문 작성에 기여했다. Hongyi Du는 milestone generation, Werewolf 프레임워크 (환경·커뮤니케이션·평가기·메모리 모듈), 데이터 분석·생성, 창발 행동·한계·관련 연구와 부록의 인간 평가, Werewolf, 주요 프롬프트, 나쁜 커뮤니케이션 사례를 담당했다. Zhaochen Hong은 환경 기본, 커뮤니케이션 모듈, 데이터베이스 환경과 관련 부록·관련 연구 작성에 기여했다. Xiaochen Yang은 메모리 모듈, Minecraft 환경과 관련 부록·관련 연구를 맡았다. Shuyi Guo는 평가기 프롬프트, bargaining 환경과 해당 부록·관련 연구를 맡았다. Zhe Wang은 reasoning agent 모듈, coding 환경과 해당 부록·관련 연구를 담당했다.

7.2 멀티 에이전트 프레임워크 설계 세부 사항

MARBLE 프레임워크는 설정, 환경, 메모리, 커뮤니케이션, 행동 모듈로 구성된다.

- Configuration Module은 과제 명세, 페르소나 데이터, 에이전트 프로필, 역할 정의, 도메인별 데이터베이스를 입력받아 에이전트 속성과 관계를 초기화한다. 에이전트 프로필에는 능력, 제약, 성격 특성이 포함되며, 에이전트 간 관계는 위계, 협업 링크, 적대 관계 등으로 정의된다.
- Environment Module은 코딩, 연구 프로젝트, 협상 게임 등 에이전트가 작동하는 시나리오를 시뮬레이션한다. 에이전트는 함수 호출 인터페이스를 통해 행동을 선택하고, 환경은 행동 결과에 따라 상태를 갱신한다. 도메인별 기능은 Tool Box가 제공한다.
- Memory Module은 공유 메모리와 개별 메모리를 저장·검색한다. 공유 메모리는 전역 지식과 집단 결정을 담고, 개별 메모리는 개인 경험과 지역 관찰을 유지한다. 개별 메모리는 장기·단기 세그먼트로 나뉘며, 단기 메모리는 FIFO 방식으로 관리된다. RAG는 동적 지식 접근과 프롬프트 구성을 돕는다.
- Communication Module은 에이전트 간 외부 상호작용을 관리한다. 각 에이전트에 커뮤니케이션 도구와 다른 에이전트의 상세 프로필을 제공하여, 역할 협상, 계획 조정, 협력·경쟁의 균형을 지원한다.
- Action Module은 에이전트가 생성한 계획을 실행하고, 함수 호출 및 구조화 출력 형식을 활용해 최종 결과를 얻는다. 행동 결과와 관찰은 즉시 개별·공유 메모리에 반영되어 반복적으로 전략을 조정한다.

7.3 인간 평가

프롬프트 기반 평가가 인간 판단과 얼마나 일치하는지 확인하기 위해 Werewolf 환경에서 인간 평가를 수행했다. NLP 연구에 익숙한 6명의 평가자가 동일한 지시와 입력을 보고 계획 및 커뮤니케이션 차원을 평가했다. 각 과제는 두 명이 평가하고 평균을 사용했으며, 총 60개 과제가 다섯 개 LLM에 대해 Werewolf 환경에서 평가되었다. Kendall, Pearson, Spearman 상관계수와 p-value를 계산해 인간 점수와 기계 점수의 정렬을 확인했다.

보고된 표에서 다섯 모델의 인간 평가 점수와 프롬프트 기반 기계 점수는 대체로 가깝다. 예컨대 커뮤니케이션 점수의 가장 큰 차이는 0.38 이내였고, 대부분의 차이는 더 작았다. 저자들은 이를 근거로 프롬프트 기반 평가가 인간 평가자가 인식하는 조정·계획 품질과 유사한 측면을 포착한다고 본다.

7.4 Research Scenario

연구 시나리오는 전문 연구 프로필을 가진 여러 에이전트가 협력해 새로운 연구 아이디어를 생성하는 과제다. 에이전트들은 완전 연결 그래프 형태로 서로 협력하며, 최종 목표는 명확성, 관련성, 실행 가능성을 보장하는 5-question(5Q) 형식의 연구 아이디어를 만드는 것이다.

환경은 문헌 탐색과 연구 아이디어 생성을 위한 도구를 제공한다. 주요 도구는 관련 논문 검색, 최신 논문 검색, 저자 출판물과 공동저자 네트워크 수집, 키워드·arXiv ID·제목 기반 논문 조회, 웹페이지 수집 등이다. 데이터셋은 ResearchTown 프로젝트에서 온 ML/AI 논문 100편으로 구성되며, 각 논문의 introduction과 저자 이력 기반 프로필을 사용한다. 난이도는 쉬운 과제 33개, 중간 과제 34개, 어려운 과제 33개로 나뉜다.

과제 완성도는 5Q 형식으로 생성된 연구 아이디어를 기준으로 평가한다. 핵심 평가 항목은 혁신성, 안전성, 실행 가능성이다. 5Q는 문제 정의, 중요성, 어려움, 기존에 해결되지 않은 이유, 접근법과 기대 결과의 핵심 구성 요소를 묻는다. 부록은 NLoTM 관련 introduction을 바탕으로 과제를 제시하는 사례, 생성 모델·이미지 처리·GAN·연합학습 등 경력을 가진 에이전트 프로필 사례, transformer 기반 악성코드 탐지 아이디어를 5Q로 생성한 사례를 포함한다. 최종 평가는 구조화된 프롬프트로 수행되며 5점 척도와 세부 피드백을 사용한다.

7.5 Werewolf Environment

Werewolf 환경은 마피아류 사회적 추론 게임에서 영감을 받은 경쟁·협력 시나리오다. 플레이어는 마을 주민 진영과 늑대인간 진영으로 나뉘며, 역할별 비대칭 정보와 목표를 갖는다. 이 환경은 숨겨진 역할, 속임수, 집단 추론, 반복 의사결정을 포함하므로 불확실성 하의 LLM 기반 멀티 에이전트 조정을 평가하는 데 적합하다.

평가는 주로 마을 주민 진영에 초점을 둔다. 주민 진영은 예언자(Seer)의 정보 공개, 마녀(Witch)의 해독제·독약 사용, 경비(Guard)의 보호, 투표 합의 등 명시적 협력이 승패에 중요하기 때문이다. 반면 늑대인간은 공개적 협력 없이도 오도와 혼란을 이용해 승리할 수 있어, 협력 잠재력 평가에는 주민 측이 더 분별력 있는 관찰 대상이라고 설명한다. 주민 모델을 바꾸는 실험에서 늑대인간 측은 항상 GPT-4o로 고정해 안정적인 도전적인 상대를 유지했다.

벤치마크는 단일하고 안정적인 초기 설정에서 시작한다. 역할은 wolf, villager, seer, witch, guard 등이며, 모든 메시지는 환경을 거치는 엄격한 이벤트 버스 방식으로 처리된다. 환경은 night start, seer action, vote action 같은 이벤트를 순차적으로 발행하고, 에이전트 응답을 적절한 시점에 전달해 Werewolf 규칙과 정보 흐름을 통제한다. 각 에이전트 관점의 이벤트 로그와 최종 결정은 재현성과 사후 평가를 위해 저장된다.

실험에는 GPT-4o 기반 에이전트가 플레이한 다양한 게임 상태 100개가 archive로 준비된다. 두 가지 모드가 있다.

- Partial-Day Simulation은 특정 밤 상태에서 시작해 정확히 하루-밤 주기를 실행한다. 주민에게는 “의심되는 늑대인간 추방”, “예언자 보호”, “늑대인간에게 독 사용”, “위협받는 주민 구출” 같은 단기 과제가 주어지고, 완료 비율을 측정한다.
- Full-Game Simulation은 첫 번째 밤 이후 상태에서 전체 게임을 끝까지 실행한다. 과제는 주민에게 제안 형태로만 제공되며, 부분 완료가 아니라 전체 process score, 최종 승패, 결과 점수를 평가한다.

Result Score는 게임 종료 시 생존 주민 수에서 생존 늑대인간 수를 뺀 값이다. Partial-Day의 일일 과제는 예언자 보호(+1), 늑대인간 추방(+2), 주민 구출(+2 및 핵심 역할이면 +1 추가), 늑대인간 독살(+2) 등으로 정의된다. 단일 일자의 이론적 최대 점수는 핵심 역할 구출 추가점을 제외하고 5점이다.

Full-game 및 single-day에서는 주민과 늑대인간 점수 누적으로 net score도 계산한다. 예를 들어 주민은 보안관 선출, 경비 보호 성공, 마녀 구출, 늑대인간 독살, 낮 투표로 늑대인간 추방, 늑대인간에게 투표한 주민 수에 따라 점수를 얻고, 주민 오투표나 마녀의 주민 독살은 감점된다. 늑대인간은 보안관 선출, 밤 공격 성공, 낮에 주민 추방 등으로 점수를 얻는다. 저자들은 full-game에서 net score가 약 5이면 주민 승리 확률이 매우 높고, 0-5에서는 결과가 흔들리며, 0 미만이면 늑대인간이 대체로 압도적으로 승리한다고 관찰한다.

Task Score는 partial-day 과제 완료율과 full-game 주민 승률을 0-100 범위로 스케일한 뒤 평균한다. Collaboration Score는 커뮤니케이션 점수와 계획 점수의 평균이다. GPT-4o가 시뮬레이션 로그, 마녀와 예언자의 내부 추론까지 읽고 각 점수를 매긴다. 인간 평가 결과는 이 프롬프트 기반 점수와 가깝다고 보고된다.

상세 결과에서 single-day completion ratio는 Llama3.3-70B가 0.3754로 가장 높고, villager net score도 유일하게 양수(0.2802)였다. full-run에서도 Llama3.3-70B는 net score 0.4511, result score -0.1915, 주민 승률 약 35.11%로 가장 강한 결과를 보였다. GPT-4o baseline은 net score -2.1946, 주민 승률 24.73%였다. Llama3.1-8B와 GPT-4o-mini는 주민 승률이 각각 약 1.15%, 3.09%로 낮았다. 저자들은 신뢰할 수 있는 투표 휴리스틱, 보호 조치, 장기 계획 통합이 중요하다고 해석한다.

사례 연구는 세 가지를 제시한다. 첫째, Llama-3.1-8B가 예언자 역할을 맡은 경우, 스스로의 결백과 역할을 반복해 주장하지만 구체적 검사 결과나 논리적 증거를 제공하지 못해 설득력을 잃고 첫날 추방된다. 둘째, Seer와 Witch가 GPT-4o인 경우에도 예언자가 첫 밤에 늑대인간을 찾기도 신분 노출을 두려워해 정보를 공개하지 않고, 마녀도 해독제 사용을 주저해 실패한다. 핵심 원인은 상호 신뢰와 협력 부족으로 분석된다. 셋째, Llama3.3-70B 주민이 GPT-4o 늑대인간을 상대한 full-game 사례에서는 경비가 첫날 밤 죽는 불리한 상황에도, 마녀의 보안관 출마·역할 공개, 예언자의 조사 결과 공개, 보안관 배지 전달을 통해 승리한다. 이는 적절한 역할 공개, 신뢰 형성, 결정적 투표권 유지가 중요함을 보여 준다.

7.6 Database Environment

Database Environment는 D-Bot에서 영감을 받은 PostgreSQL 데이터베이스 성능 이상 진단 환경이다. Docker에서 실행되는 PostgreSQL에 정상 SQL 쿼리를 먼저 실행해 다양한 상황을 시뮬레이션한 뒤, 부적절한 쿼리로 이상을 발생시킨다. 에이전트들은 그래프 구조로 배치되어 서로 대화하고, pg_locks, pg_stat_statements 같은 시스템 뷰를 질의해 데이터베이스 동작과 성능 정보를 확인한다.

다루는 이상 유형은 Fetch Large Data, Insert Large Data, Lock Contention, Redundant Index, Vacuum의

다섯 가지다. Auto-vacuum은 기본 활성화되어 있으나 Vacuum 원인 사례에서는 특정 테이블의 auto-vacuum을 끄고 수동 vacuum을 사용한다. 실험에서는 실제 root cause를 1개로 제한하지만, 에이전트는 가장 가능성 높은 root cause 2개를 예측할 수 있다.

난점은 여러 이상 징후가 동시에 관찰될 수 있지만 모두 root cause는 아니라는 점이다. 예컨대 Fetch Large Data를 위해서는 데이터 삽입이 선행되므로 Insert Large Data가 함께 나타날 수 있고, 100개 스레드가 같은 테이블에 접근하기 때문에 Lock Contention도 관찰될 수 있다. 정상 쿼리와 문제 쿼리가 섞여 있는 점도 난도를 높인다.

테스트셋은 10개 시뮬레이션 시나리오로 구성된다: E-Commerce, Education, File-sharing, Finance, Healthcare, Internet of Things, Manufacturing, Music Streaming, Social Media, Transportation. D-Bot과의 차이는 5개 에이전트를 사용하고, planner가 각 에이전트에게 가능한 root cause 하나씩을 탐색하도록 할당한다는 점이다. 에이전트는 어떤 테이블을 질의해야 하는지에 대한 프롬프트는 받지만, 실행할 구체 쿼리에 대한 외부 지식이나 결과 분석 도구는 없다. 과제 점수는 50개 테스트 샘플에 대한 예측 정확도를 5점 척도로 스케일한다. 예측한 두 root cause 중 하나가 정답이면 correct로 본다.

7.7 Coding Scenario

Coding Scenario는 서로 보완적인 코딩 능력을 가진 에이전트들이 구조화된 프로그래밍 과제를 협력해 해결하는 환경이다. 에이전트는 디버깅, 코드 실행, 테스트 작성 등 특정 영역에 특화되어 있으며, 목표는 요구사항에 맞고 정확하며 모듈화된 고품질 솔루션을 만드는 것이다.

도구는 create_solution, execute_code, give_advice, revise_code, code_debugger, write_test_case, review_code 등 소프트웨어 개발 생애주기의 여러 단계를 지원한다. 벤치마크는 SRDD 데이터셋을 각색해 만들었고, Education, Work, Life, Game, Creation의 다섯 주제를 포함한다. LLaMA-3-70B-instruct를 사용해 원래 SRDD 지시에서 영감을 얻고, 코딩 도메인의 네 가지 조정 전략을 반영했다: adaptive task execution, dependency management, cross-domain collaboration, test-driven development.

평가 기준은 지시 준수, 실행 가능성, 일관성, 품질이다. 기본 점수는 1-5점 척도이며, 3점 이상인 솔루션만 보너스 단계로 넘어가 결함 없는 실행, 혁신적 해결, 모범적 코딩 관행에 대한 추가 점수를 받을 수 있다.

7.8 Bargaining Scenario

Bargaining Scenario는 다자 협상을 통해 실제 의사결정 과정을 모사하는 환경이다. 각 에이전트는 성격, 목표, 우선순위, 전략이 다른 협상 프로필을 부여받는다. 환경에서는 두 판매자와 두 구매자가 상호작용하며, 각자 목표를 달성하기 위해 가격과 조건에 반응한다. 이 시뮬레이션은 경쟁적 목표와 협력적 의사결정의 균형을 평가한다.

도구는 offer_price, reject_and_counter, accept_offer, provide_information, inquire_intentions, end_negotiation 등이다. 데이터셋은 Amazon 제품 데이터에서 무작위로 샘플링한 100개 제품을 기반으로 하며, 제품명, 원가, 할인가, 사용자 평점 등 협상에 필요한 속성을 포함한다. 각 에이전트에는 Big Five 성격 프로필이 부여되고, GPT 기반 모델로 성격과 역할에 맞춘 협상 전략을 생성한다.

제품 가격은 5.80달러에서 149.99달러까지 분포하며 평균은 30.71달러다. 평점 평균은 3.97, 표준편차는 1.44이며, 78개 고유 카테고리를 포함한다. 협상 스타일은 aggressive, cooperative, neutral 중에서 무작위로 선택된다.

구매자는 가격 협상, 배송 시간, 제품 품질, 서비스 유연성을 중시하고, 판매자는 재고 소진, 브랜드 평판, 반복 구매, 대량 할인을 중시한다.

평가는 전략의 효과성, 진행과 결과, 상호작용 동역학을 기준으로 5점 척도에서 수행된다. 부록의 예시는 One Happy Camper High Chair Banner(14.99달러, 평점 4.8/5)를 둘러싼 협상이다. 구매자는 가격과 품질의 균형을 추구하고, 판매자는 프리미엄 가격을 정당화한다.

상세 협업 점수에서 최종 Bargaining Score는 buyer와 seller 역할 점수를 평균해 계산한다. 보고된 결과에서는 GPT-4o-mini가 3.710으로 가장 높은 최종 Bargaining Score를 얻었다. Task-based 점수에서도 seller 점수가 모든 모델에서 buyer 점수보다 일관되게 높았으며, 저자들은 모델이 구매자로서 가격을 깎는 것보다 판매자로서 높은 가격을 정당화하고 제안을 방어하는 일을 더 쉽게 할 수 있음을 시사한다고 해석한다. GPT-4o-mini는 buyer 3.578, seller 3.869로 양쪽 모두에서 가장 높았다.

7.9 Minecraft Scenario

Minecraft 환경의 과제는 에이전트가 주어진 구조 설명에 따라 블록 구조물을 만드는 것이다. 각 구조물은 특정 위치와 방향의 블록들로 구성된다. 현재 모델 능력에 맞추기 위해 몇 가지 단순화가 적용된다. 설명에는 필요한 위치·방향·블록 종류가 모두 포함되고, 필요한 블록은 에이전트 출생지 근처 컨테이너에 제공되며, 이동 가능 영역이 제한되고, 공격적 생물은 제거된다. 최종 성능은 올바른 유형·위치·방향으로 배치된 블록의 hit rate로 평가된다.

환경은 VillagerAgent를 각색했고, Mineflayer를 엔진으로 사용해 텍스트 기반 Minecraft 상호작용을 가능하게 한다. VillagerAgent의 40개 이상 도구 중 건축 과제와 관련된 11개 도구만 사용한다. 여기에는 주변 엔티티 스캔, 특정 위치 이동, 블록 채굴, 블록 배치, 아이템 장착, 블록 전달, 컨테이너에서 아이템 꺼내기, 흙 사다리 설치·해체, 컨테이너 내용 확인, 환경 정보 조회 등이 포함된다.

테스트 사례 역시 VillagerAgent에서 가져온 100개 목표 구조물이다. 필요한 블록 수 분포는 대체로 균형적이며, 블록 수가 많을수록 과제 난도가 높다고 본다. Task Completion Metric은 $\text{Hit_rate} = \#(\text{Matched_block}) / \#(\text{Total_block}) \times 100\%$ 로 계산한다. 평가 시 에이전트는 목표 구조물 설명을 받고 협력해 과제를 수행하며, 상호작용 턴 수에는 상한이 있다. 각 테스트 끝에서 hit rate를 5점 척도로 매핑하고, 상호작용과 자기 계획 단계도 5점 척도의 협업 점수로 평가한다.

결과 분석에서는 모든 다섯 모델에서 필요한 블록 수가 늘어날수록 성능이 감소했다. 이는 과제 난도가 높아질수록 모델들이 취약해진다는 의미다. 특히 Llama-3.1-70B의 task score가 매우 낮았는데, 원인은 함수 호출 실행 가능률이 다른 모델보다 현저히 낮기 때문으로 분석된다. 두 GPT 모델은 거의 100% 실행 가능한 함수 호출을 보였고, 다른 두 Llama 모델은 약 80%였으나, Llama-3.1-70B는 절반 미만이었다.

7.10 Execution-Based Milestone Evaluation

이 방식에서 milestone은 과제 실행 중 동적으로 식별된다. 에이전트는 미리 정의된 평가 지표와 피드백 루프를 사용해 진행 상황을 실시간 추적하며, 특정 하위 목표가 달성되었다고 판단되면 해당 milestone을 완료로 표시한다. 변화하는 과제 조건에 적응할 수 있어 불확실성이 높거나 창발적 도전이 있는 시나리오에 적합하다.

7.11 Predefined Milestone Generation

Predefined milestone은 과제 실행 전에 생성된다. 먼저 LLM에 상세한 과제 설명을 제시하고, 구체적 목표와 산출물을 가진 구조화된 milestone으로 과제를 분해하도록 프롬프트를 설계한다. 이후 단계별 추론을 통해 과제 세분화를 반복적으로 다듬고, 각 milestone은 이름, 목적, 하위 과제, 기대 결과를 포함하는 구조화된 dictionary로 표현된다. 이 과정은 GPT-4 기반 반복 개선과 전문가 검토를 활용해 고품질 과제 분해를 보장한다.

7.12 Important Prompts

부록은 멀티 에이전트 협업과 과제 결과 평가에 쓰인 핵심 프롬프트를 제시한다. 대표 프롬프트는 Collaboration Score(communication and planning), Research Task Score(5Q), KPI Prompt다. Werewolf처럼 환경별 특수 프롬프트는 수가 많아 생략되었다.

Communication Evaluation Prompt는 과제, 에이전트 프로필, 사회적 관계, 집계된 과제 결과와 커뮤니케이션 데이터를 입력으로 받아, 의사결정 효과성, 명확성과 정밀성, 사회적 관계 준수, 프로필 정렬, 전반적 과제 진척 기여도를 1-5점으로 평가한다. 출력은 JSON의 score 필드다.

Planning Evaluation Prompt는 에이전트 프로필과 모든 iteration의 planning data를 입력으로 받아, 과제 할당의 명확성, 역할 정의, workload distribution, 결과 효과성, 전략적 조정을 평가한다. 마찬가지로 1-5점 척도와 JSON score 출력을 요구한다.

KPI Prompt는 연구 과제 iteration에서 milestone이 달성되었는지 판단한다. milestone 유형은 의미 있는 5Q를 처음 구성한 form 5q 또는 이전 iteration 대비 유의미하게 개선한 improve 5q로 나뉜다. 출력에는 milestone_achieved, milestone_type, 핵심 기여 에이전트 2-3명이 포함된다.

Task Score(5Q) Evaluation Prompt는 최종 연구 아이디어의 혁신성, 안전성, 실행 가능성을 평가한다. 유효한 5Q 답변을 구성할 수 없으면 세 항목 모두 1점을 부여하고, 여러 5Q가 있으면 가장 최근 응답을 평가한다. 출력은 innovation, safety, feasibility 점수를 담은 JSON이다.

7.13 Bad Communication Cases

부록은 멀티 에이전트 시스템에서 나쁜 커뮤니케이션이 협업을 어떻게 방해하는지 보여 주는 사례를 제시한다. 문제 유형은 과도한 반복, 실질적 진전 부족, 무의미한 상호작용, 역할 혼란이다.

첫 번째 사례는 연구 아이디어 논의에서 두 에이전트가 tensor decomposition, anomaly detection, random projection, Tensor Train, Rademacher distribution 등을 언급하지만, 대부분 상대의 말을 반복하고 구체적 방법이나 실행 계획으로 나아가지 못하는 대화다. 분석은 과도한 반복, 실질적 진전 부족, 자기 자신에게 지시하는 듯한 비효율적 교환, 응답 다양성 부족, 역할·과업 배분 부재를 지적한다.

두 번째 사례는 Minecraft에서 smooth sandstone을 찾고 보조 블록을 설계하는 과정이다. 에이전트들은 컨테이너나 상자에 가자는 계획과 “함께 협력하자”는 문장을 반복하지만, 누가 어디로 이동하고 어떤 도구를 실행하며 어떤 순서로 블록을 배치할지 명확히 나누지 않는다. 이후 15개 iteration 동안 커뮤니케이션이 없다고 보고된다. 저자들은 겉으로는 협력처럼 보이지만 구체적 전략과 역할 배분이 없으면 전체 목표의 진전이 거의 없음을 강조한다.

실무적 해석: foundation model/BERT 관점

BERT 계열 모델을 포함한 foundation model 실무자에게 이 부록의 핵심은 “모델 단일 성능”과 “멀티 에이전트 실행 성능”이 다르다는 점이다. MultiAgentBench는 모델이 지식을 알고 답하는지뿐 아니라, 역할을 나누고, 도구를 호출하고, 정보를 공개하거나 숨기며, 다른 에이전트의 행동에 적응하는지를 평가한다. 이는 encoder 중심의 BERT류 모델을 파이프라인 구성요소로 사용할 때도 중요하다. 예컨대 검색·분류·이상탐지·코드검증 모듈이 개별적으로 좋아도, 공유 메모리, 라우팅, 커뮤니케이션 프로토콜, 평가 프롬프트가 잘못 설계되면 시스템 수준 성능은 떨어질 수 있다.

또한 부록의 사례들은 멀티 에이전트 평가에서 로그 기반 진단이 필수임을 보여 준다. Werewolf의 실패 사례는 정보 공개 타이밍과 신뢰 부족이 고성능 모델에서도 치명적일 수 있음을 보여 주고, Minecraft 사례는 함수 호출 실행 가능률이 task score를 크게 좌우함을 보여 준다. 따라서 실무 평가에서는 최종 점수뿐 아니라 실행 가능률, 반복·공회전 커뮤니케이션, 역할 할당의 명확성, milestone 달성 여부를 함께 기록해야 한다.